

Authentication and Access Control

Q1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Ans: Procedure

Step 1: Create Project Files

Create a project folder and add the following files:

- index.html
- style.css
- script.js

Step 2: Design the Login Interface

Create the login page using HTML. Add the following components:

- Zomato Admin Panel heading
- Username text field
- Password text field
- Login button

Step 3: Apply Styling

Use CSS to:

- Create a red and white Zomato-themed interface.
- Style the login form with proper spacing, colors, and rounded corners.
- Make the page responsive and visually appealing.

Step 4: Implement Authentication

Write JavaScript code to:

- Store hardcoded administrator credentials.
- Compare the entered username and password with the stored credentials.
- Display "**Access Granted**" if both credentials match; otherwise display "**Access Denied**".

Step 5: Run the Application

Open the project using **VS Code Live Server** (or open index.html in a web browser).

Step 6: Test the Login Form

- Enter the correct credentials (admin / admin123) and verify that **Access Granted** is displayed.
- Enter incorrect credentials and verify that **Access Denied** is displayed.

Figure 1: Zomato Admin Login Form

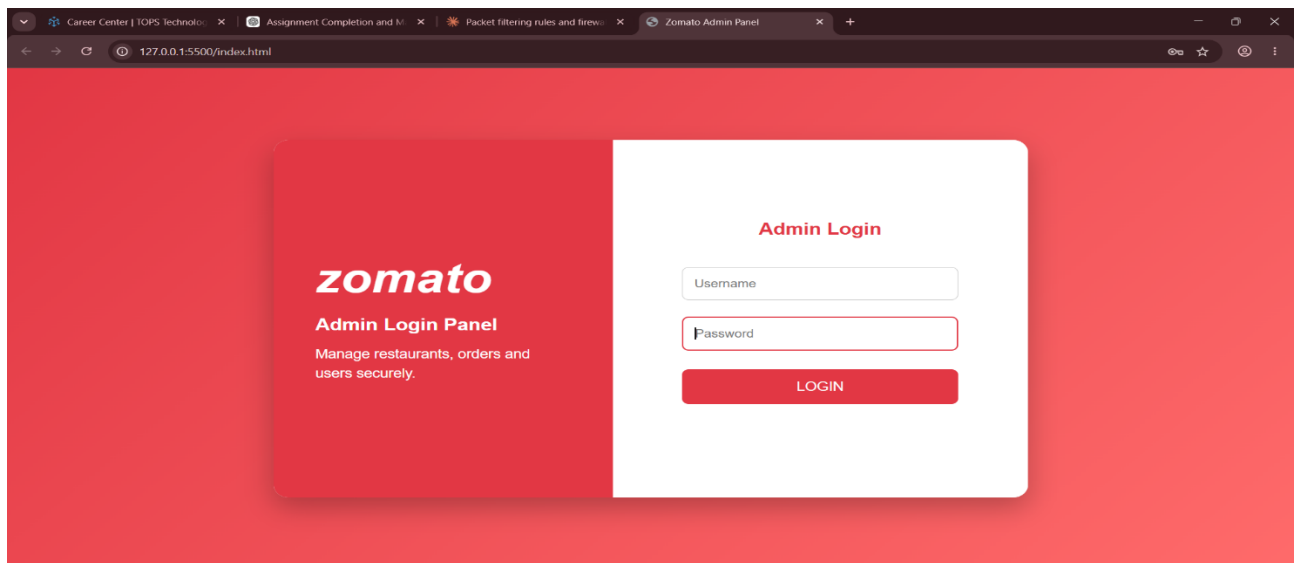


Figure 2: Successful Login – Access Granted

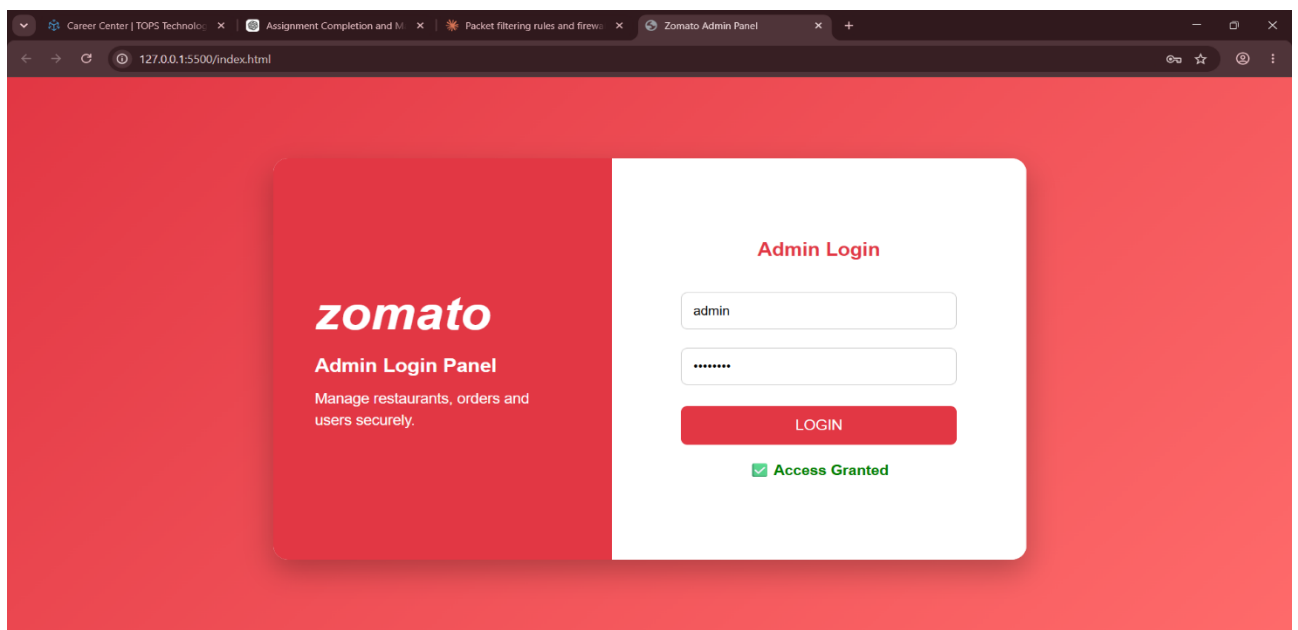
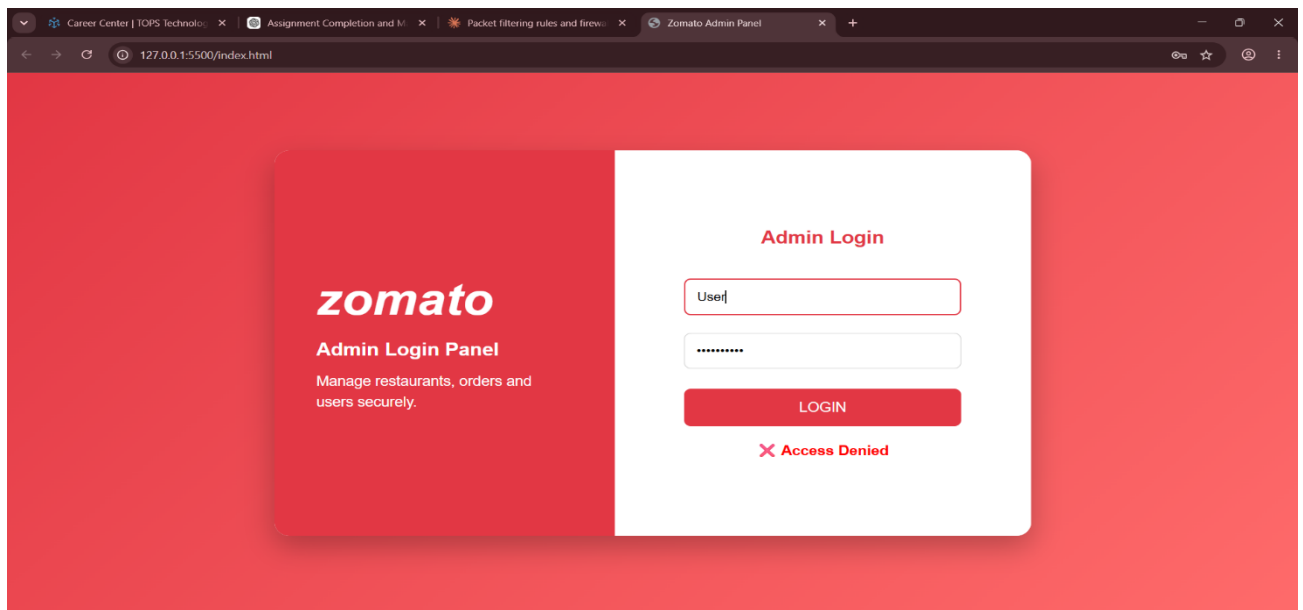


Figure 3: Invalid Login – Access Denied



Result

The mock Zomato Admin Login Panel was successfully created. The application authenticates the user using hardcoded administrator credentials and grants access only when the correct username and password are entered.

Q2. Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Hint: Use a dictionary to map roles to allowed resources.

Ans: Figure 1 : Python Program

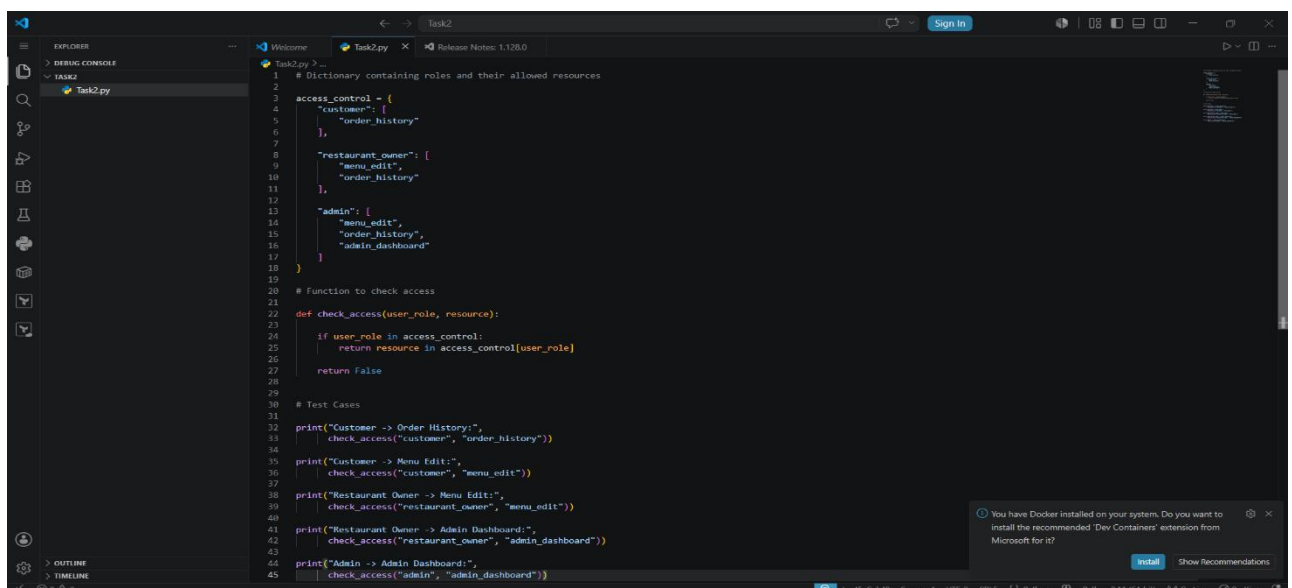
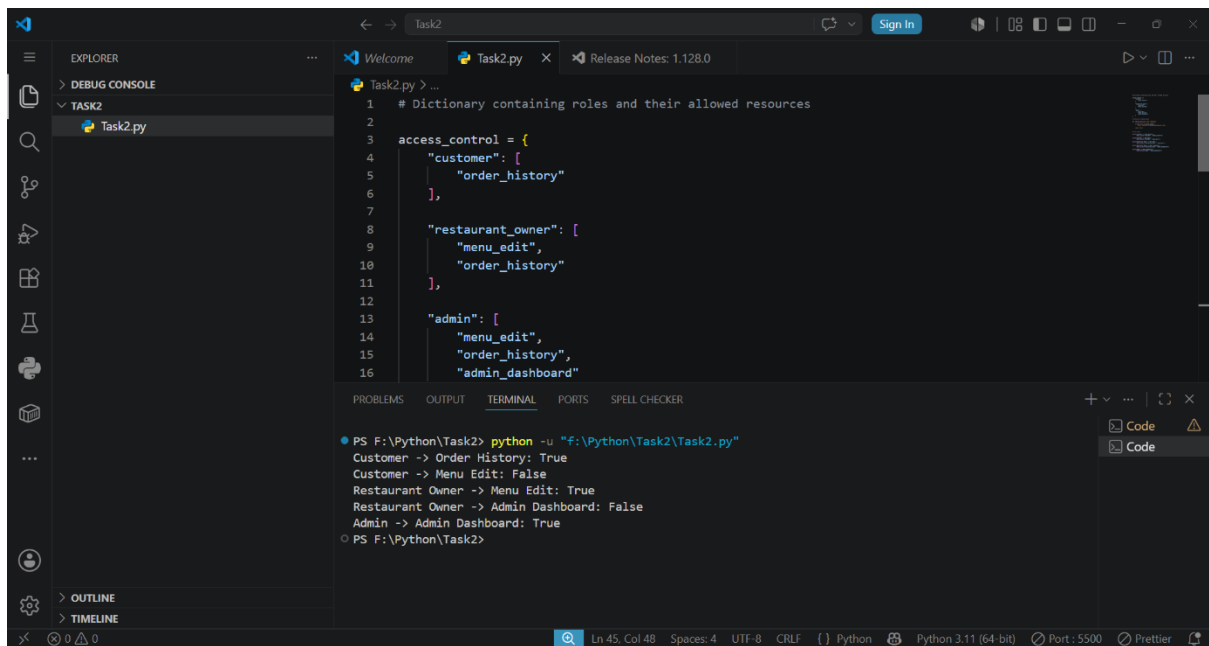


Figure 2: Output of the Python Program



The screenshot shows a Visual Studio Code editor with a Python file named `Task2.py`. The script defines an `access_control` dictionary with three roles: `customer`, `restaurant_owner`, and `admin`, each with a list of allowed resources. The terminal output shows the results of running the script, confirming the permissions for each role.

```
1 # Dictionary containing roles and their allowed resources
2
3 access_control = {
4     "customer": [
5         "order_history"
6     ],
7
8     "restaurant_owner": [
9         "menu_edit",
10        "order_history"
11    ],
12
13    "admin": [
14        "menu_edit",
15        "order_history",
16        "admin_dashboard"
17    ]
18 }
```

```
PS F:\Python\Task2> python -u "f:\Python\Task2\Task2.py"
Customer -> Order History: True
Customer -> Menu Edit: False
Restaurant Owner -> Menu Edit: True
Restaurant Owner -> Admin Dashboard: False
Admin -> Admin Dashboard: True
PS F:\Python\Task2>
```

Explanation

The program uses a Python dictionary to define which resources each user role can access.

The `check_access()` function checks whether the requested resource exists in the list of resources assigned to the given user role. If permission is available, the function returns **True**; otherwise, it returns **False**.

The program was tested using three different roles:

- **Customer**
- **Restaurant Owner**
- **Admin**

The output confirms that only authorized users are allowed to access specific resources.

Test Results

User Role	Resource	Result
Customer	Order History	True
Customer	Menu Edit	False
Restaurant Owner	Menu Edit	True

Restaurant Owner	Admin Dashboard	False
Admin	Admin Dashboard	True

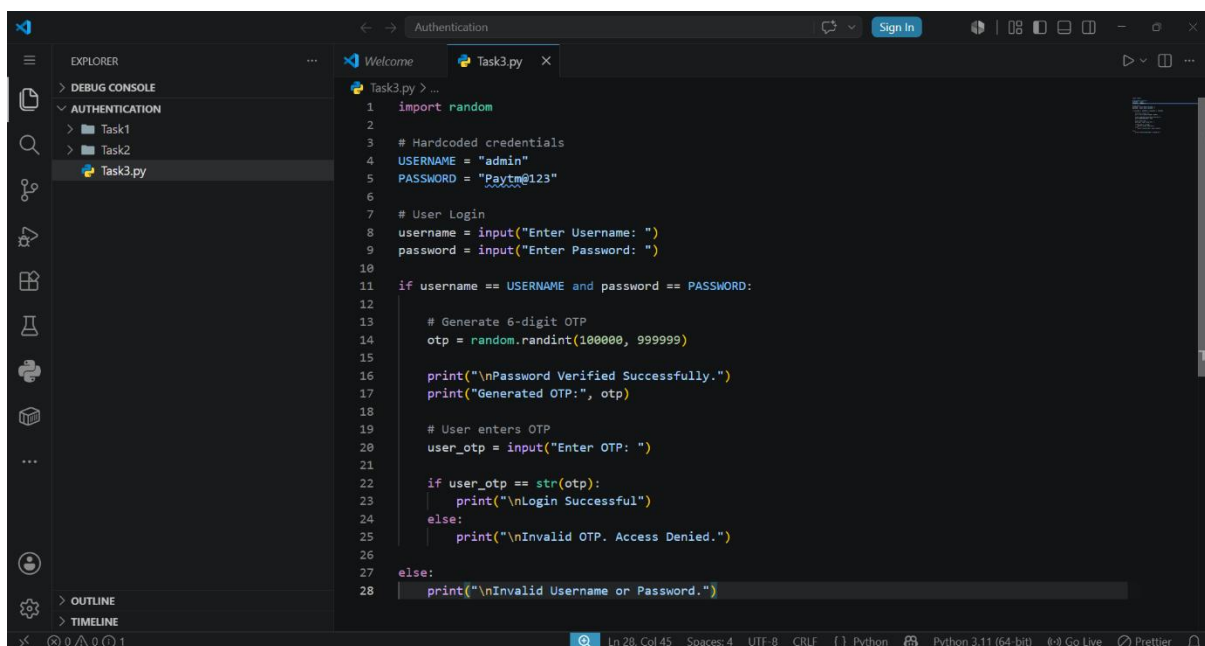
Conclusion

The `check_access()` function successfully implements **Role-Based Access Control (RBAC)** using a Python dictionary. It returns **True** when the user has permission to access the requested resource and **False** when access is not permitted.

Q3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Ans:

Figure 1: Python Program

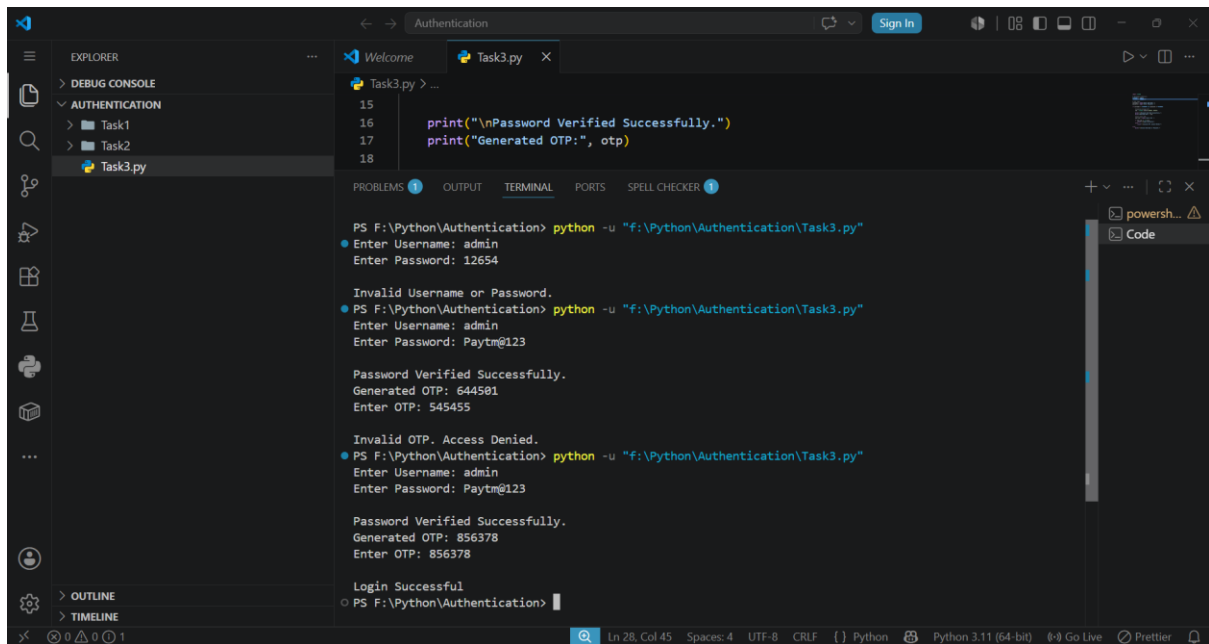


```

1  import random
2
3  # Hardcoded credentials
4  USERNAME = "admin"
5  PASSWORD = "Paytm@123"
6
7  # User Login
8  username = input("Enter Username: ")
9  password = input("Enter Password: ")
10
11 if username == USERNAME and password == PASSWORD:
12     # Generate 6-digit OTP
13     otp = random.randint(100000, 999999)
14
15     print("\nPassword Verified Successfully.")
16     print("Generated OTP:", otp)
17
18     # User enters OTP
19     user_otp = input("Enter OTP: ")
20
21     if user_otp == str(otp):
22         print("\nLogin Successful")
23     else:
24         print("\nInvalid OTP. Access Denied.")
25
26 else:
27     print("\nInvalid Username or Password.")
28

```

Figure 2: Successful Two-Factor Authentication using Python



```
15 print("\nPassword Verified Successfully.")
16 print("Generated OTP:", otp)
17
18
```

```
PS F:\Python\Authentication> python -u "f:\Python\Authentication\Task3.py"
Enter Username: admin
Enter Password: 12654

Invalid Username or Password.
PS F:\Python\Authentication> python -u "f:\Python\Authentication\Task3.py"
Enter Username: admin
Enter Password: Paytm@123

Password Verified Successfully.
Generated OTP: 644501
Enter OTP: 545455

Invalid OTP. Access Denied.
PS F:\Python\Authentication> python -u "f:\Python\Authentication\Task3.py"
Enter Username: admin
Enter Password: Paytm@123

Password Verified Successfully.
Generated OTP: 856378
Enter OTP: 856378

Login Successful
PS F:\Python\Authentication>
```

Explanation

The program first verifies the username and password using hardcoded credentials. If the credentials are correct, it generates a random 6-digit OTP using Python's random module and displays it in the console.

The user must enter the generated OTP. If the entered OTP matches the generated OTP, the program displays **"Login Successful"**; otherwise, it displays **"Invalid OTP. Access Denied."** If the username or password is incorrect, the OTP is not generated and access is denied immediately.

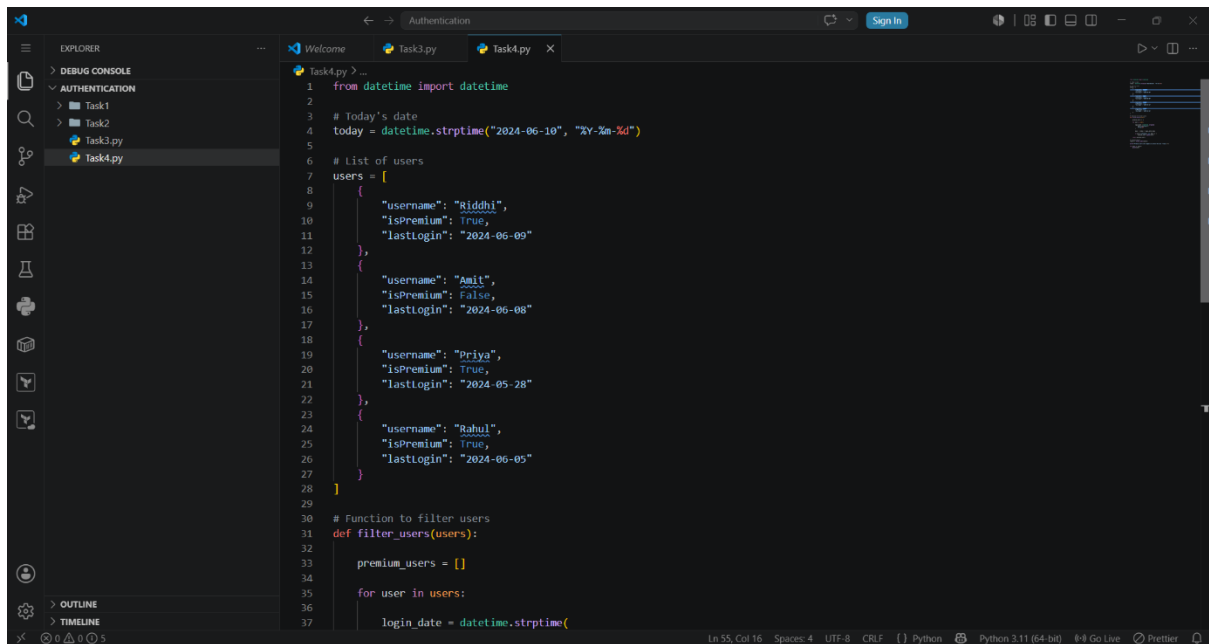
Conclusion

The program successfully simulates a basic Two-Factor Authentication (2FA) process. It demonstrates how an additional OTP verification step improves security by requiring both the correct password and a valid one-time password before granting access.

Q4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

Ans:

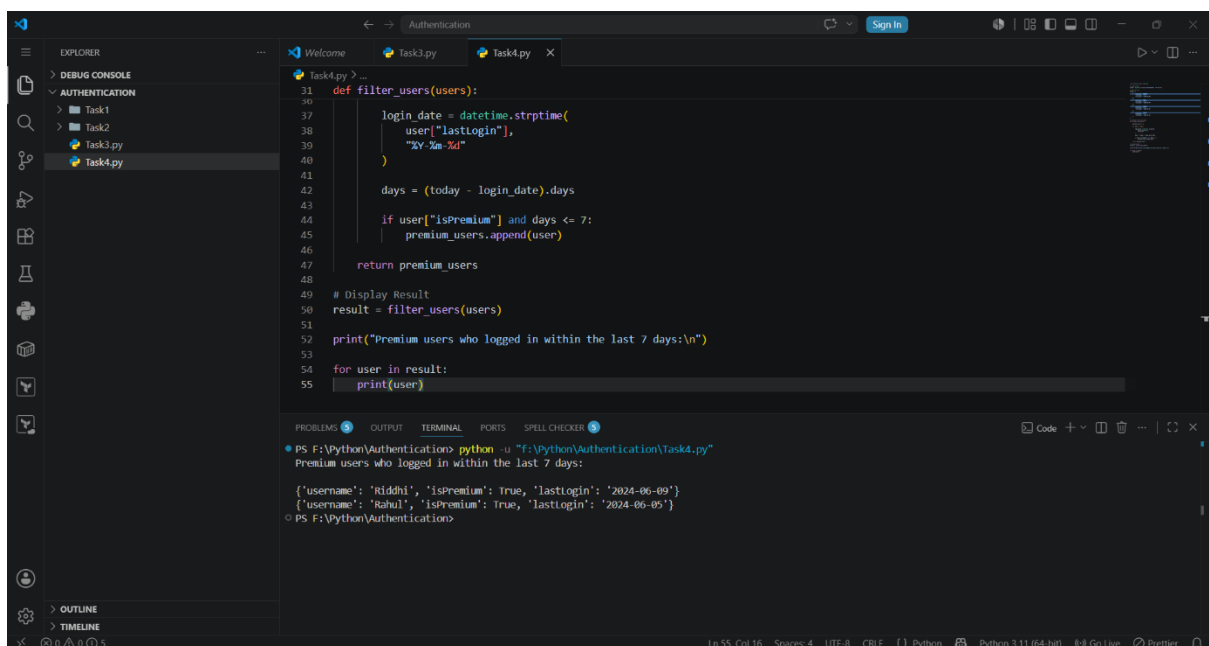
Figure 1: Python Program



This screenshot shows the first part of a Python script in VS Code. The Explorer panel on the left shows a project named 'Authentication' with files 'Task1.py', 'Task2.py', 'Task3.py', and 'Task4.py'. The main editor displays the code for 'Task4.py'.

```
1 from datetime import datetime
2
3 # Today's date
4 today = datetime.strptime("2024-06-10", "%Y-%m-%d")
5
6 # List of users
7 users = [
8     {
9         "username": "Riddhi",
10        "isPremium": True,
11        "lastlogin": "2024-06-09"
12    },
13    {
14        "username": "Amit",
15        "isPremium": False,
16        "lastlogin": "2024-06-08"
17    },
18    {
19        "username": "Priya",
20        "isPremium": True,
21        "lastlogin": "2024-05-28"
22    },
23    {
24        "username": "Rahul",
25        "isPremium": True,
26        "lastlogin": "2024-06-05"
27    }
28 ]
29
30 # Function to filter users
31 def filter_users(users):
32
33     premium_users = []
34
35     for user in users:
36
37         login_date = datetime.strptime(
```

Figure 2: Output of Premium User Filtering Program



This screenshot shows the second part of the Python script in VS Code. The Explorer panel is the same. The main editor displays the code for 'Task4.py'.

```
31 def filter_users(users):
32
33     login_date = datetime.strptime(
34         user["lastlogin"],
35         "%Y-%m-%d"
36     )
37
38     days = (today - login_date).days
39
40     if user["isPremium"] and days <= 7:
41         premium_users.append(user)
42
43     return premium_users
44
45 # Display Result
46 result = filter_users(users)
47
48 print("Premium users who logged in within the last 7 days:\n")
49
50 for user in result:
51     print(user)
```

The terminal at the bottom shows the command prompt and the output of the script:

```
PS F:\Python\Authentication> python -u "F:\Python\Authentication\Task4.py"
Premium users who logged in within the last 7 days:
{'username': 'Riddhi', 'isPremium': True, 'lastlogin': '2024-06-09'}
{'username': 'Rahul', 'isPremium': True, 'lastlogin': '2024-06-05'}
PS F:\Python\Authentication>
```

Explanation

The program stores user information in a list of dictionaries. It converts each user's last login date into a date object and calculates the difference between today's date and the login date.

The `filter_users()` function checks whether a user is a premium member and has logged in within the last seven days. Users satisfying both conditions are returned and displayed.

Conclusion

The Python program successfully filters users based on premium membership and recent login activity. This approach can be used in real-world applications to identify active premium users for personalized services, notifications, or security checks.

Q5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Ans:

Access Control Mistake	Fix / Prevention Tip
1. Weak or Default Passwords	Enforce strong password policies and encourage multi-factor authentication (MFA).
2. Improper Role-Based Access Control (RBAC)	Grant users only the permissions required for their role (Principle of Least Privilege).
3. Missing Authorization Checks	Verify user permissions on every request before allowing access to protected resources.
4. Insecure Session Management	Use secure session tokens, enable automatic session expiration, and invalidate sessions after logout.
5. Exposed Admin Pages or APIs	Restrict access to administrative pages and APIs using authentication, authorization, and firewall rules.

Conclusion

Applications such as **Instagram** and **Flipkart** use proper authentication, role-based access control, secure session management, and firewall protection to prevent unauthorized access and protect user data from security threats.